



System design

Image Carousel ★ Premium



Medium



30 mins



1.48k done

Report an issue 🛠

Asked at these companies



Airbnb



Amazon



Dropbox



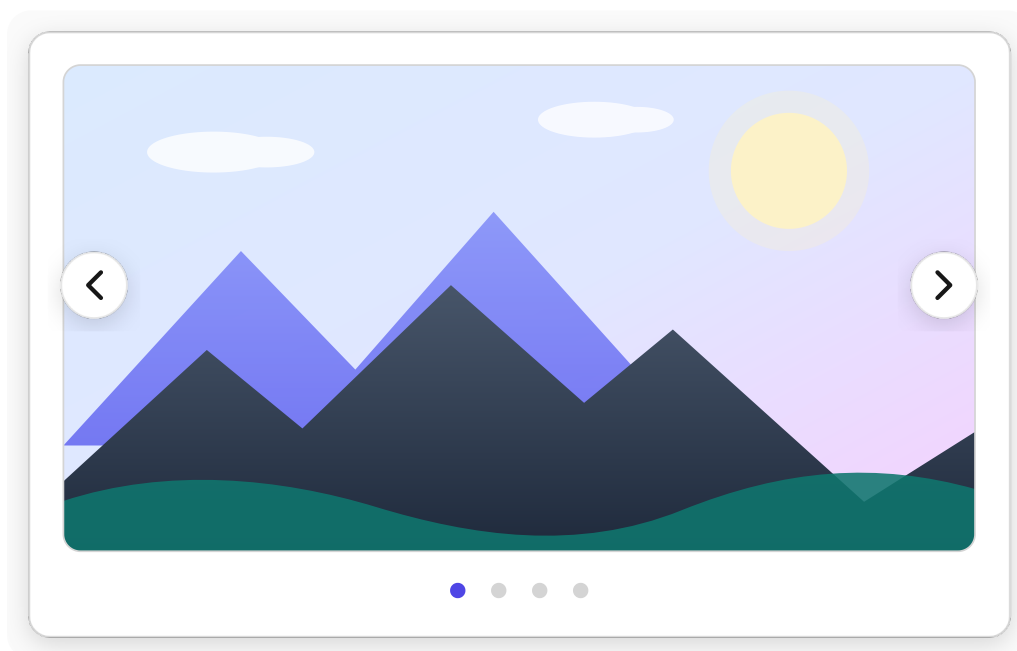
Google



Microsoft

Question

Design an image carousel component that displays a list of images one at a time, allowing the user to browse through them with pagination buttons.



Requirements exploration

Before designing the API, we clarify how images are supplied, which devices the carousel must support, and which navigation and customization controls are in scope.

How are the images specified?

It will be a configuration option on the component, and the list of images has to be specified before initializing the component.

What devices should the component support?

Desktop, tablet, and mobile.

How will the pagination buttons behave when the user is at the start/end of the image list?

It should cycle through infinitely.

Will there be animation when transitioning between images?

Yes, the images should animate with horizontal translation.

Exercise Beta

Which requirements define the scoped image carousel before its architecture is chosen? Select all that apply.

- ☐ 1 Cycle from the final image to the first and from the first image to the final one.
- ☐ 2 Animate image changes with horizontal movement.
- ☐ 3 Support desktop, tablet, and mobile interaction.
- ☐ 4 Require consumers to own the active index and replace every slide and control renderer.

5

Receive the complete image list as component configuration before initialization.

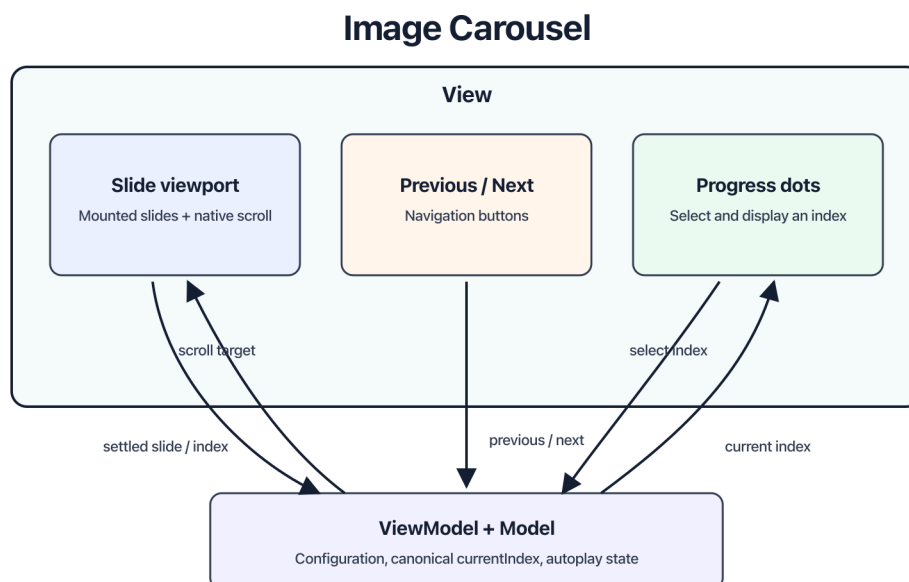


Reveal answer

Check answer ↩

Architecture / high-level design

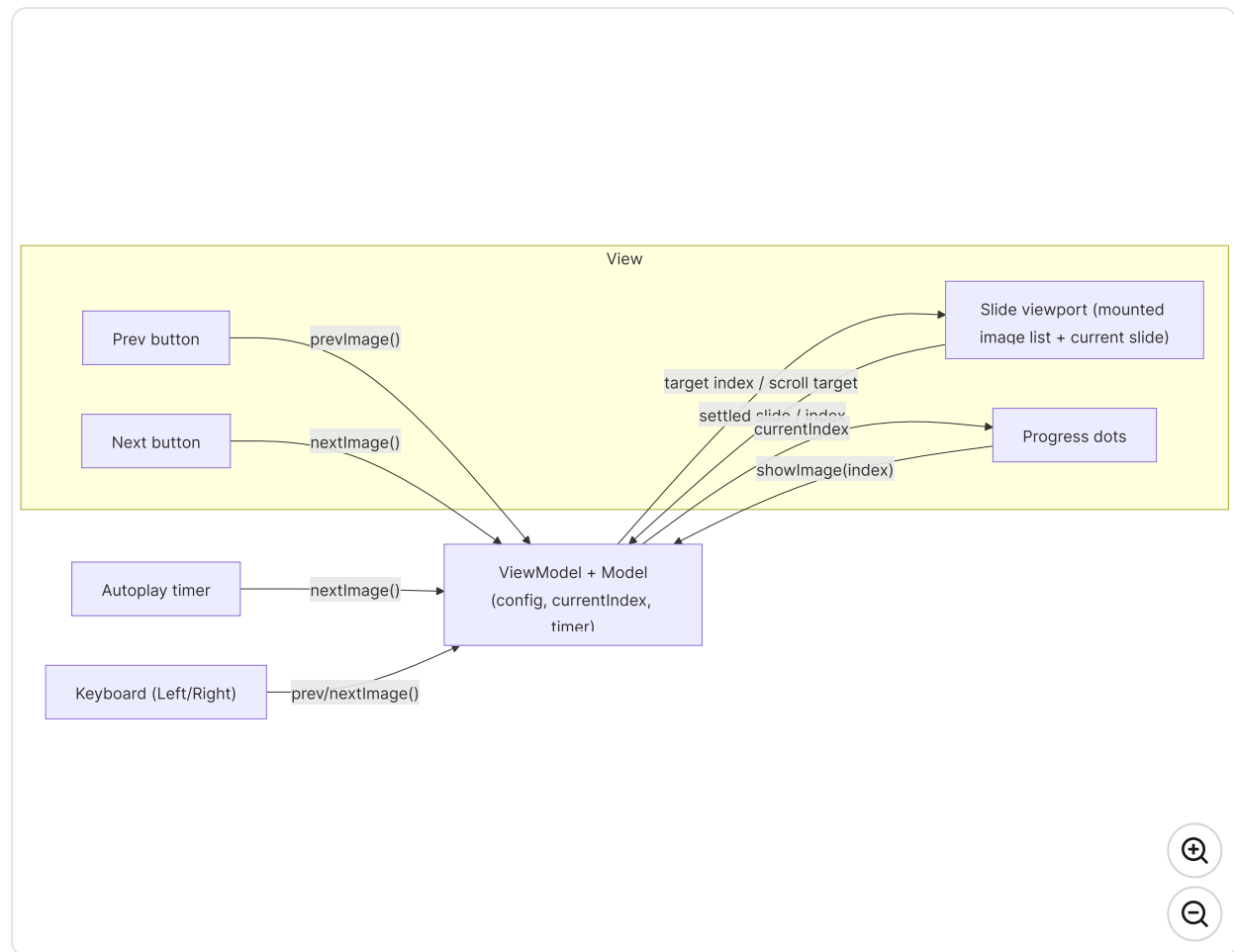
Since this component doesn't need any server data, our architecture will just consist of the client-side components.



Component responsibilities

- **ViewModel + Model:** The brain of the component; holds the configuration and state of the component, orchestrates events between the components, and tracks which image is current.
- **Slide viewport:** Contains the mounted image list in a horizontal scroll container and exposes the currently selected slide.
- **Prev/Next Buttons:** Tell the "ViewModel" to change to the prev/next image depending on which button is clicked.
- **Progress Dots:** Tell the "ViewModel" which image to show when the respective dot/page is clicked/selected.

There's no need to separate the "Model" and the "ViewModel" in this component because it's a small component.



Component hierarchy and action flow

Reducer-style controller

A small reducer-style controller is enough to separate action sources from transition logic. Buttons, swipes, a timer, and key presses can all dispatch the same previous, next, or select actions without requiring an application-wide state library.

Exercise Beta

Which architecture best supports several carousel presentations using the same navigation behavior?

- 1 Persist raw `scrollLeft` as canonical state and replay that pixel value after every resize.
- 2 Keep index and transition rules in a controller while controls and slide renderers consume that state.

3

Let the track own a transform index, pagination own a selected index, and each slide report visibility independently.



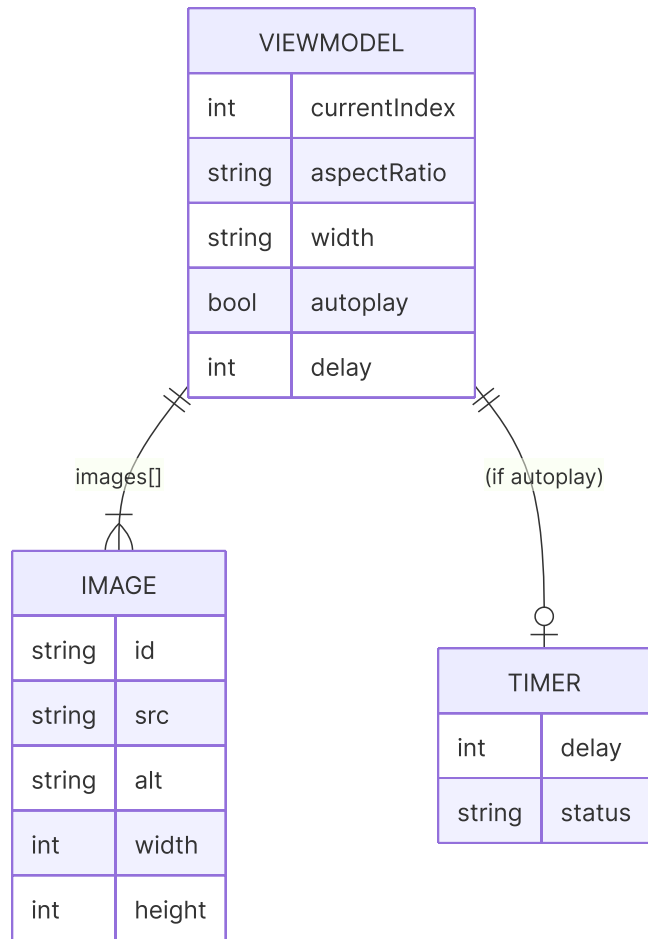
Reveal answer

Check answer ↩

Data model

Only the "ViewModel" will hold any state and data; the other components are part of the view and won't hold data. It can contain the following fields:

- Configuration
 - List of images, including a stable ID, URL, `alt` value, and intrinsic dimensions.
 - Size: Responsive width and aspect ratio of the carousel.
- State
 - Index of current image. Controls request a target index, while the settled viewport position commits the canonical value after button, keyboard, swipe, or autoplay navigation.
 - Autoplay timer and pause state, if autoplay is enabled.

*ViewModel state and configuration relationships*

Exercise Beta

A responsive carousel accepts a fixed image list and commits navigation after scrolling settles. Which values belong in its logical model? Select all that apply.

1

The active slide's stable identity or index.

☐

2

Store mounted slide elements as the canonical image collection.

☐

3

An ordered list of slides with stable IDs and media metadata.

☐

4

Persist the current transform offset and reconstruct the active slide from pixels after reload.

☐

5

The carousel's responsive width and aspect-ratio configuration.

☐[Reveal answer](#)[Check answer ↩](#)

Interface definition (API)

Since we're talking about designing a UI component here, the API will focus on the external API of the components: what configuration options are provided so that developers can use the component in a customized fashion.

Basic API

- **List of images:** An array of stable slide records with image URLs, required `alt` values, and intrinsic dimensions; responsive sources can be included when available.
- **Height / aspect ratio:** A responsive CSS size or aspect ratio rather than a required fixed pixel height.
- **Width:** A CSS size such as `100%` or a maximum width rather than a required fixed pixel width.

An example of an `ImageCarousel` component defined in React:

```
1  <ImageCarousel
2    images=[
3      {
4        id: "foo",
5        src: "https://example.com/images/foo.jpg",
6        alt: "A foo",
7        width: 1600,
8        height: 1000,
9      },
10     {
11       id: "bar",
12       src: "https://example.com/images/bar.jpg",
13       alt: "A bar",
14       width: 1600,
15       height: 1000,
16     },
17     /* More images if desired. */
18   ]
19   aspectRatio="8 / 5"
20   width="100%"
21 />
```

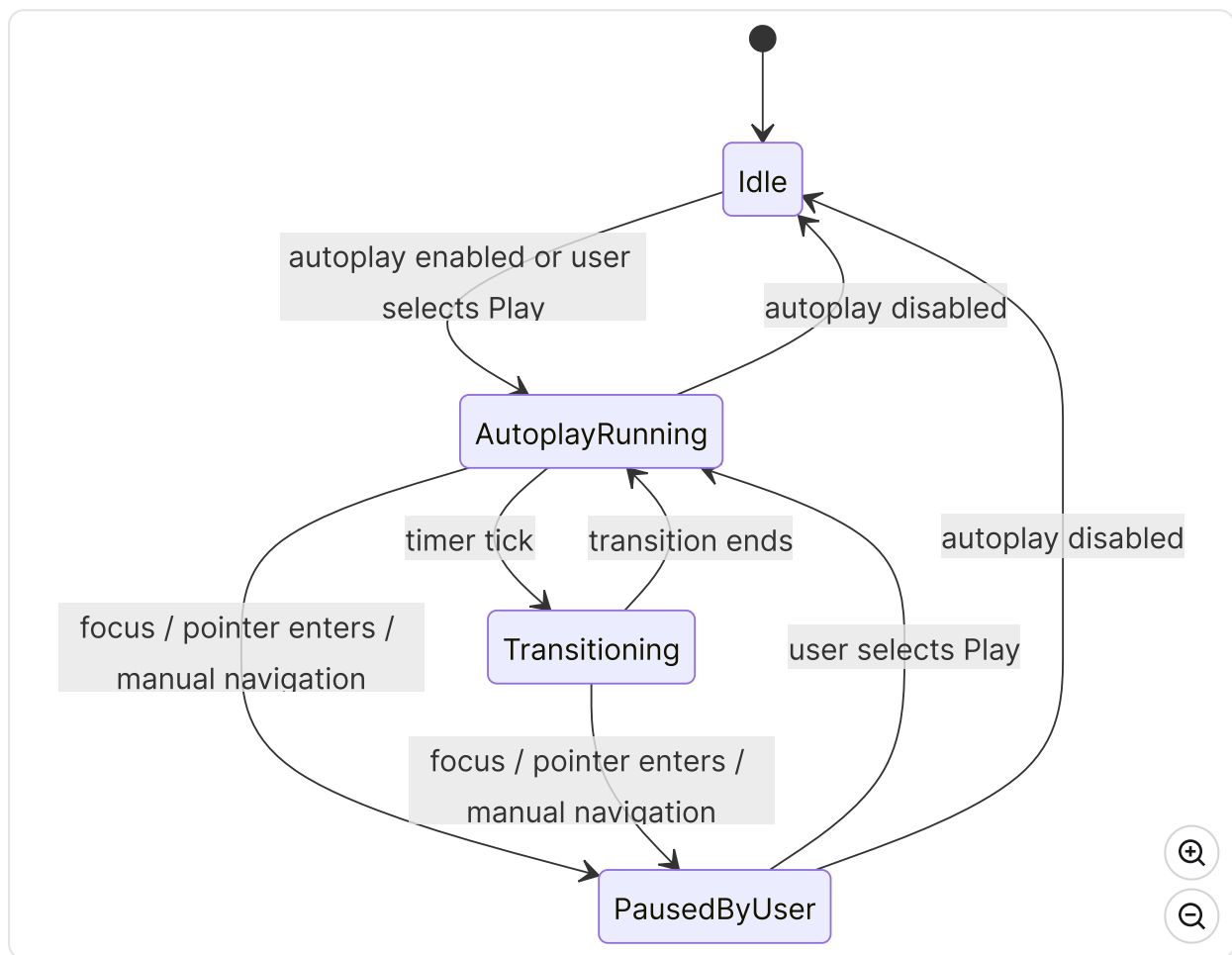
Advanced API

These are non-essential APIs but worth a discussion if there's time.

- **Autoplay:** Whether the carousel will automatically progress to the next image after some time.
 - A timer state value will be needed to keep incrementing the image.
 - Manual navigation or focus entering the carousel stops autoplay until the user explicitly restarts it.
- **Delay:** Delay between transitions. Only needed if the carousel is in autoplay mode.
- **Transition duration (ms):** Only applies to a transform-driven alternative. Native smooth-scroll timing is controlled by the browser.
- **Event listeners:** It'd be useful to add event listeners to buttons of the component so that developers can implement their own extra functionality (e.g. logging user interactions).
 - **onLoad** : When the first image is done loading and shown in the carousel.

- **onIndexChange** : When a settled active slide changes, with the target index and source (`previous` , `next` , `dot` , `swipe` , or `autoplay`).
- **Theming**: See Front End Interview Guidebook's UI Components API Design Principles Section.
- **Loop**: Defaults to `true` for the scoped requirement; a reusable component can allow consumers to disable it.

Autoplay needs a small state machine so that focus, pointer interaction, or manual navigation stops the timer until the user explicitly selects Play again.



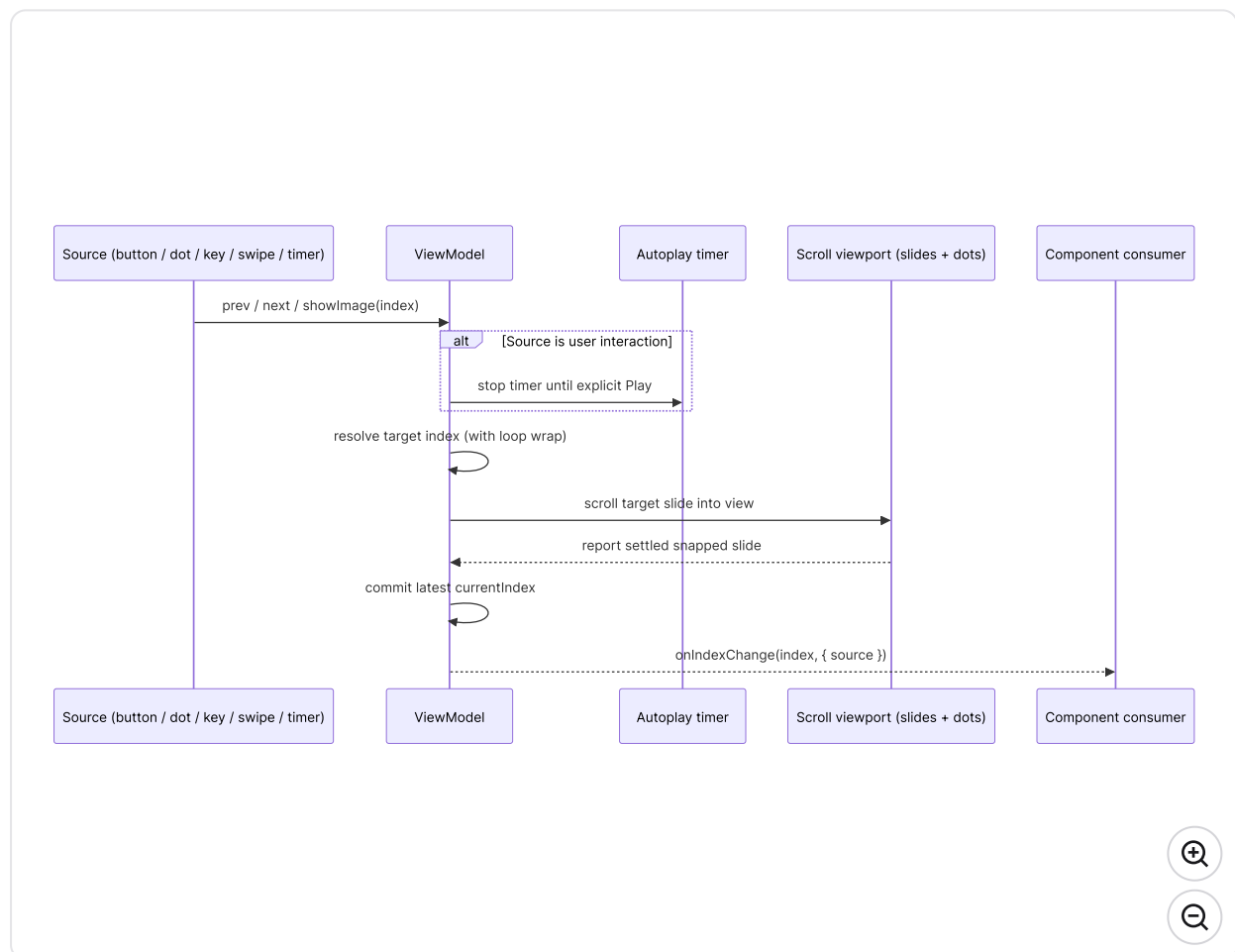
Autoplay lifecycle with user interruption

Internal API

API	Source	Description
<code>prevImage()</code>	Prev Button / Keyboard / swipe	Shows the previous image

API	Source	Description
<code>nextImage()</code>	Next Button / Keyboard / swipe / autoplay	Shows the next image
<code>showImage(index)</code>	Progress Dots / controlled updates	Skips to a specific image

- These behaviors are encapsulated within APIs because they can be called from multiple places (UI elements, timers) and can contain a fair amount of logic depending on whether looping behavior is turned on.
- `prevImage()` and `nextImage()` can call `showImage(index)` internally.
- These internal APIs can be reducer actions. Redux is only useful when the carousel state genuinely belongs to application-wide state.



Navigation action flow from multiple sources

Only the latest settled navigation request commits and fires `onIndexChanged` ; an interrupted request or a finite-boundary no-op does not emit a change.

Exercise 1 of 2 Beta

Which external API gives the baseline carousel enough information to render stable, responsive images?

- 1 Accept stable image records with sources, alternative text, intrinsic dimensions, and responsive sizing.
- 2 Accept image URLs plus one required pixel width and height for every viewport and source.
- 3 Accept prebuilt slide DOM elements and derive image metadata from their attributes during navigation.
- 4 Accept only image URLs and measure each rendered element after it loads to discover identity and size.



Reveal answer

Check answer ↩

Optimizations and deep dive

The deeper discussion centers on the details that make a carousel feel right across devices, media sizes, input methods, and languages:

- **Layout implementation**: This section explains how to arrange slides, animate transitions, and position navigation controls.
- **User experience**: This section covers scroll snapping, pagination behavior, touch gestures, autoplay expectations, and interaction details.
- **Performance**: This section discusses image loading, preloading, formats, sizing, and bandwidth control for media-heavy carousels.
- **Internationalization (i18n)**: This section covers localized control labels and text-direction considerations for carousel navigation.
- **Accessibility (a11y)**: This section explains keyboard support, screen-reader behavior, focus handling, and safer carousel patterns.

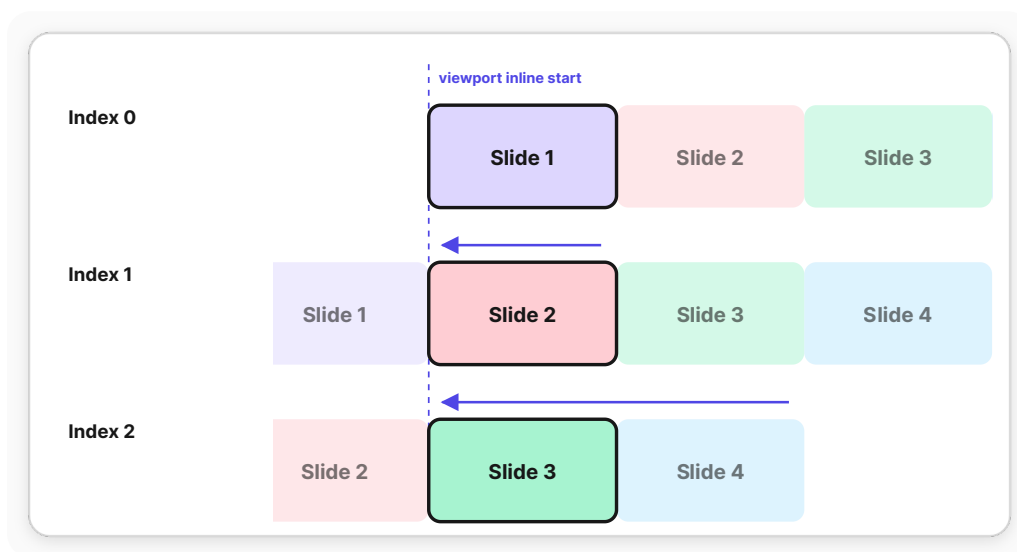
Layout implementation

The core layout decisions are how to arrange the image strip, how to animate transitions between slides, and how to position the navigation controls.

Images

A simple version to achieve the layout is to use `display: flex` to make the images render in a horizontal row and programmatically change the horizontal scroll offset to show the various images.

You can draw out such a diagram on the whiteboard to illustrate the layout:



The outlined box indicates the currently visible window.

Sample Code

```
1 <div class="carousel-images">
2   
3   
4   
5   <!-- More images -->
6 </div>
```

```
1 .carousel-images {
2   display: flex;
```

```
3   height: 100%;
4   width: 100%;
5   overflow-x: auto;
6   scroll-snap-type: x mandatory;
7 }
8
9 .carousel-image {
10  flex: 0 0 100%;
11  height: 100%;
12  width: 100%;
13  scroll-snap-align: start;
14 }
```

To scroll to a particular image smoothly, we can do:

```
1  document.querySelectorAll(".carousel-image")[selectedIndex].scrollIntoView
2    behavior: "smooth",
3    block: "nearest",
4    inline: "start",
5  });
```

Scrolling the target element into view avoids hard-coding slide width and respects the container's text direction. After scrolling settles, derive `currentIndex` from the snapped slide so touch, keyboard, and button navigation share one source of truth.

Re-align the active slide after its container resizes; exact observer wiring is a production follow-up.

Fitting Images

The layout above assumes that all the images are of the same size. However, it is unlikely that the user will provide images that are of that exact size.

Here are a few ways we can get around this:

CSS `background-size` : This requires a change in the HTML to render the image using CSS `background` / `background-image` instead of ``. The advantage of this is that we can use the `background-size` property that has these two modes:

- `contain` : Scales the image as large as possible within its container without cropping or stretching the image. If the container is larger than the image, this will result in image tiling, unless the `background-repeat` property is set to `no-repeat`.
- `cover` : Scales the image (while preserving its ratio) to the smallest possible size to fill the container (that is: both its height and width completely cover the container), leaving no empty space. If the proportions of the background differ from the element, the image is cropped either vertically or horizontally.

Source: [background-size - CSS: Cascading Style Sheets | MDN](#)

Both options have their merits, and which to use really depends on the provided images. One way is to allow the developer to customize whether to use `contain` or `cover` for all the images, or even allow customizing this setting for individual images.

CSS `object-fit` : CSS `object-fit` is a feature that is similar to how `background-size` works for `background-image` s, but for `<img>` and `<video>` HTML tags. It has `contain` and `cover` properties as well, which work the same way as `background-size` 's version.

This way is preferred since it's more semantic to use `<img>` tags than `<div>` s with `background-image` s.

✓ **Prefer `<img>` with `object-fit` over `<div>` backgrounds for slides**

`<img>` keeps the slide semantic for assistive tech, exposes alt text, and benefits from native lazy loading and srcset. CSS `background-image` should only be reached for when you genuinely need background-specific behavior that `object-fit` cannot replicate.

Vertically-center buttons

To vertically center the buttons, we can use `position: absolute` on the buttons along with `transform: translateY(-50%)` to shift them up by half their height.

```
1 <div class="carousel-image-container">
2   <div class="carousel-images">
3     
4     
```

```
5     <!-- More images -->
6   </div>
7   <button class="carousel-button carousel-button-prev">...</button>
8   <button class="carousel-button carousel-button-next">...</button>
9 </div>
```

```
1  .carousel-image-container {
2    aspect-ratio: 8 / 5;
3    width: 100%;
4    position: relative; /* So that position: absolute will be relative to
5  }
6
7  .carousel-button {
8    position: absolute;
9    top: 50%;
10   transform: translateY(-50%); /* Shifts the button up by half its height
11 }
12
13 .carousel-button-prev {
14   inset-inline-start: 30px;
15 }
16
17 .carousel-button-next {
18   inset-inline-end: 30px;
19 }
```

Exercise 1 of 2 Beta

A responsive native-scroll carousel shifts while images load and sometimes reports the wrong active slide after a swipe. Which choices address both problems? Select all that apply.

- ☐ 1 Give every slide the full responsive viewport width and a stable scroll-snap position.
- ☐ 2 Reserve each slide's expected aspect ratio before its image loads.
- ☐ 3 Recalculate every snap point from DOM measurements during each scroll event.

4

Measure the active image after decoding and animate the container from its previous height.

☐

5

Derive the active index from the slide where native scrolling settles.

☐

Reveal answer

Check answer ↩

User experience

- **Scroll snapping:** Use CSS scroll snapping and show the scrollbar so that users can still use native scrollbars to scroll through the images but the images will "snap" nicely and the scroll position will nicely align to show a full image once scrolling stops. Mobile users expect to be able to swipe through the images and CSS scroll snap helps you achieve this on mobile devices easily.
- **Interactive elements should be large enough.** The Prev/Next buttons should be at least 44px tall and wide to be easy to tap on mobile devices. One trick is to increase the hit area of the button to the entire leftmost/rightmost section. The dots are probably too small on mobile for precise interactions and can just be hidden or made non-interactive.
- **Reposition Prev/Next buttons:** It's more convenient to have the Prev/Next buttons close to each other. This goes against the example design but speeds up navigation because the distance between the buttons is minimized.



Default to CSS scroll snap for swipe and native scrolling on mobile

A horizontal scroller with `scroll-snap-type: x mandatory` and `scroll-snap-align` on each slide gives you touch swiping, momentum, and keyboard-accessible scrolling essentially for free. Reach for JavaScript-driven transforms only when you need behavior that scroll snap cannot express.

For looping, use an explicit jump between the first and last slides. Seamless clone-and-reset looping is a potential follow-up. When `loop={false}`, keep the current slide stable at either boundary and consistently disable or hide the unavailable action. With zero or one slide, hide navigation and autoplay.

A baseline looping carousel is on its final image and receives a Next action. Which transition preserves one navigation contract for buttons, keys, and swipes?

- 1 Resolve Next directly to the first real image and commit that image's logical index when navigation settles.
- 2 Move the first image record to the end of the canonical array whenever Next crosses the boundary.
- 3 Let swipes stop at the boundary while buttons and keys wrap directly between real indexes.
- 4 Ignore Next at the boundary but let Previous wrap from the first image to the final image.

[Reveal answer](#)[Check answer ←](#)

Performance

Carousels can be bandwidth-heavy, so the main levers are deferring off-screen image loads and choosing sensible image formats and sizes.

Defer loading of images that aren't on-screen

Most images in the carousel are never shown to the user, especially those at the back. It'd be a waste of bandwidth to load all the images if they are not being shown.

Load the first slide eagerly and give the likely LCP image appropriate fetch priority. Later slides can use `loading="lazy"`, but that attribute is a browser hint rather than an exact guarantee about when a request starts.

Preloading of images

JavaScript can be used if fine-grained control over image loading is desired. To help minimize the scenario where an image needs to be shown but hasn't finished loading, the component can pre-emptively load the next image with the following code:

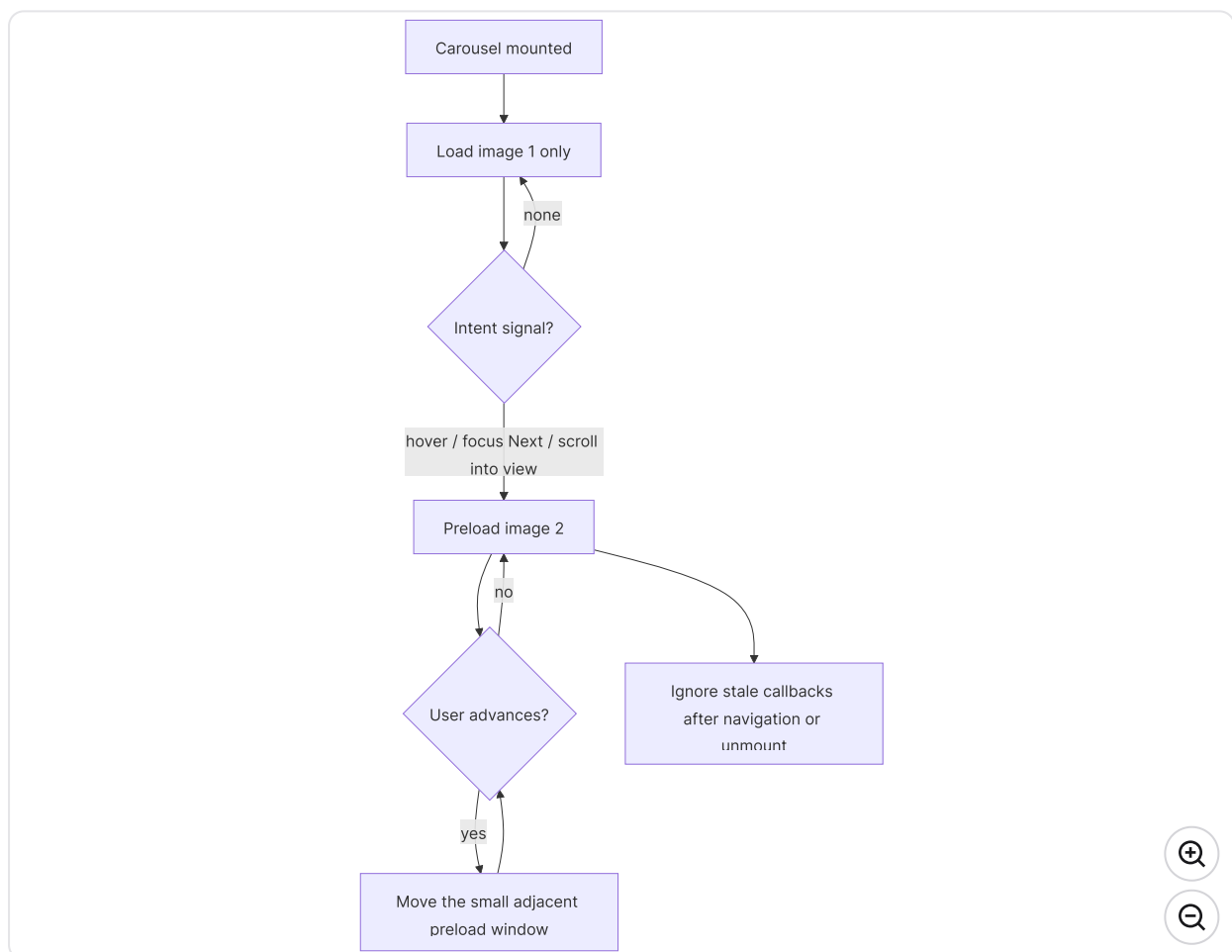
```
const preloadImage = new Image();
```

```
preloadImage.src = url;
```

Airbnb optimized its listing carousels with delayed, staggered, and cancelable image preloading:

1. Initially, later carousel images remain deferred.
2. When a listing stays about 50% visible for one second, preload the second image.
3. Cancel pending preload tasks if the carousel leaves the viewport or the user navigates away.
4. After a swipe, preload the next three images, starting each request 100ms after the previous one.

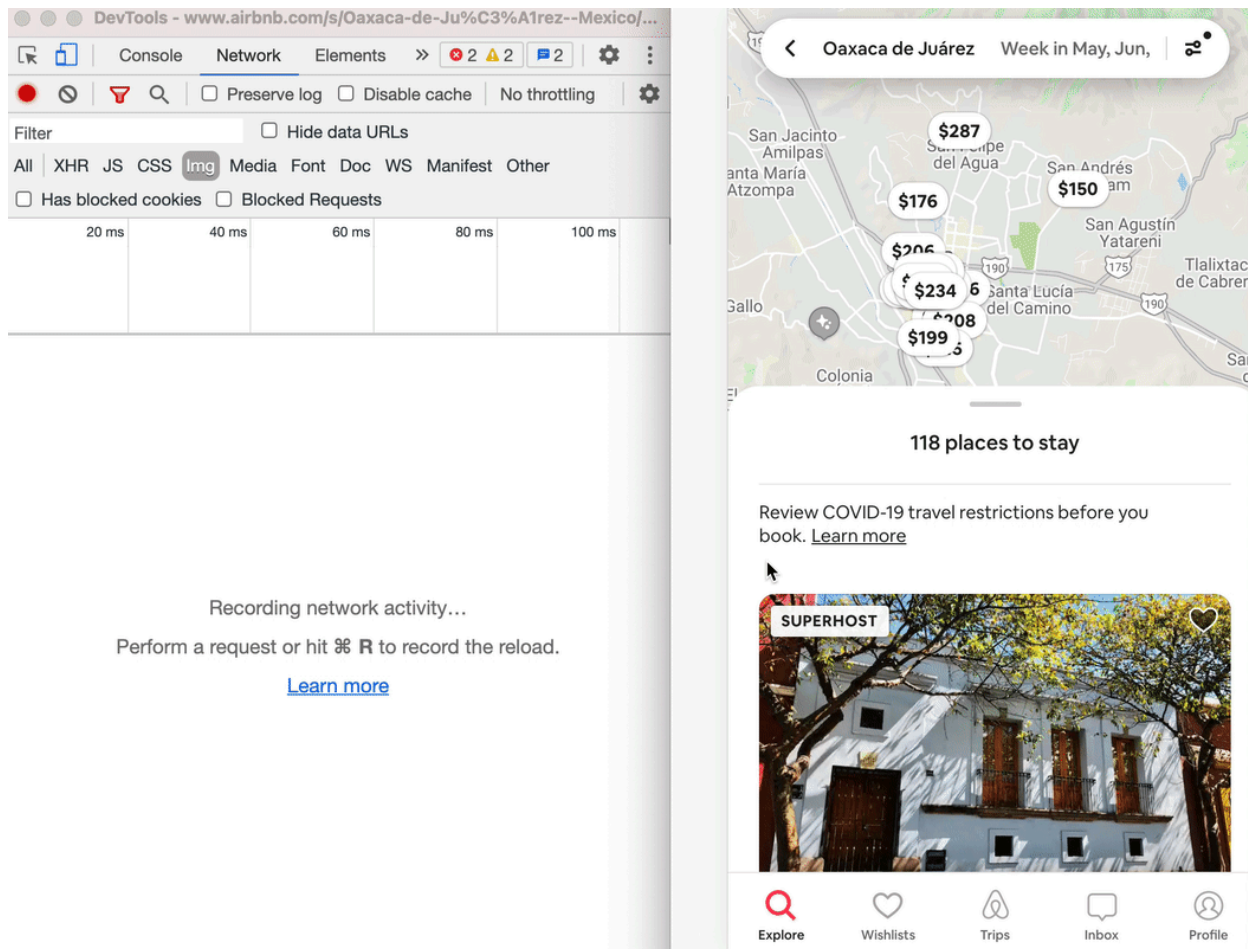
For a simpler interview baseline, preload only a small adjacent window when intent is clear, such as focus on the next button or the carousel entering the viewport. Continued navigation can advance that window without fetching the whole gallery.



Preload scheduling escalating with browse intent

✅ Preload adjacent slides on intent, not the entire image list upfront

Eagerly fetching every carousel image wastes bandwidth because most images are never viewed. Load the first slide immediately, then preload the next one or two when the user hovers, focuses the Next button, or the carousel scrolls into view, escalating only when continued navigation signals real intent.



Airbnb image carousel lazy loading example on mobile

Device-specific images

Avoid loading desktop-sized assets into a narrow carousel. Let consumers provide `srcset` and `sizes` (or `<picture>` sources for art direction) so the browser chooses among stable responsive candidates. Include matching `sizes` and intrinsic `width / height`; load the first slide eagerly while distant slides can use `loading="lazy"`.

Virtualized lists

Large-gallery virtualization is a potential follow-up rather than part of the baseline design.

A gallery contains dozens of responsive high-resolution images, but most sessions view only the first few. Which policies balance startup and next-slide latency? Select all that apply.

- ☐ 1 Provide `srcset` and `sizes` so the browser selects an appropriate asset.
- ☐ 2 Prefetch or preload a small number of likely next slides rather than the entire gallery.
- ☐ 3 Load the active image eagerly when it is important to the current view.
- ☐ 4 Download every source at a moderate resolution first, then upgrade the active slide after rendering.
- ☐ 5 Use one high-resolution source and rely on CSS to scale it to each slide size.

[Reveal answer](#)[Check answer ↩](#)

Internationalization (i18n)

i18n is not extremely relevant to this question, but control labels and slide-position status should be customizable. In RTL, inherit `dir`, use logical control placement, and target slide elements instead of assuming how physical `scrollLeft` behaves. The exact mirroring and caption policy can be a follow-up.

Exercise Beta

A carousel runs in RTL with translated controls and slide-status labels. Which behaviors should adapt? Select all that apply.

- ☐ 1 Navigate to slide elements rather than calculating targets from an assumed physical `scrollLeft` direction.
- ☐ 2 Localize button labels but keep the live slide-position announcement in English for consistency with analytics.
- ☐ 3 Translate previous, next, pagination, and slide-status labels.

4

Assume every RTL browser maps a slide by negating the LTR `scrollLeft` value.

☐

5

Place controls with logical inline positioning so the layout follows document direction.

☐

 Reveal answer Check answer ↩

Accessibility (a11y)

Image carousels are notorious for having poor accessibility due to how much effort it takes to build them right. For this reason, you should probably not build your own custom image carousels without expecting to dedicate a significant amount of time to achieve a top-quality component. Some things to take note of when building accessible image carousels:

! Rolling your own carousel almost always ships with a11y regressions

Custom carousels routinely miss keyboard traversal, focus management, screen reader labeling, and reduced-motion handling. Prefer a well-tested library unless you are committing real time to validating the component against assistive tech.

Mobile-friendliness

- Interactive elements should be large enough for mobile (at least 44px x 44px).
- Enable swiping to scroll through images (this is already the case with an `overflow-x: scroll` + CSS scroll snap implementation).
- Progress dots should be spaced further apart or made non-interactive.

Screen readers

- `<img>` tags should have a meaningful `alt` description specified or use an empty string.
- Add `aria-label`s to the Prev/Next buttons since there's no visible label within them.

- Give the carousel a labeled region with `aria-roledescription="carousel"`. Each slide can use `role="group"`, `aria-roledescription="slide"`, and a localized position label such as "2 of 5".
- Render pagination controls as labeled buttons and mark the active one with `aria-current="true"`.
- Hide inactive image-only slides from the accessibility tree. Announce a settled user-initiated change politely, but keep automatic rotation quiet.

Keyboard support

- Use the `<button>` HTML tag for the Prev/Next buttons where possible so that the buttons are focusable.
- Add `<div role="region" aria-label="Image Carousel" tabindex="0">` to make the carousel focusable and attach Left/Right keydown handlers to allow scrolling through the images with the keyboard.

When autoplay is enabled, place a Play/Pause rotation control before the rotating content in the tab order. Autoplay must stop when focus or the pointer enters the carousel or the user navigates manually, and it must restart only through that control. Disable autoplay and non-essential transitions when `prefers-reduced-motion` requests less motion, and clear timers when the component unmounts or autoplay is disabled.

Exercise 1 of 3 Beta

An image carousel must expose its controls, media, and navigation changes accessibly. Which practices keep it operable without disrupting the page? Select all that apply.

1 Make arrow and pagination controls pointer-clickable but remove them from the tab order to reduce stops. ☐

2 Give informative images contextual alternative text and decorative images empty alt text. ☐

3 Use labeled buttons for previous, next, and pagination controls. ☐

4

Expose slide position and announce user-initiated changes without flooding a live region.

☐

5

Put the slide container in an assertive live region and announce initialization, autoplay, and every intermediate transition.

☐

Reveal answer

Check answer ↩

Summary

Keep the carousel simple by concentrating coordination in one controller. The `ImageCarousel` exposes stable image records, responsive sizing, optional autoplay, and one settled `onIndexChange` contract. Previous/Next Buttons, Progress Dots, keyboard arrows, touch swipes, and the autoplay tick converge on the same three actions, `prevImage()`, `nextImage()`, and `showImage(index)`.

Keep layout close to the platform. A `display: flex` strip inside an `overflow-x: auto` viewport and `<img>` tags sized via `object-fit` let arbitrary aspect ratios land cleanly. Navigation layers on top rather than replacing defaults: absolutely positioned Prev/Next Buttons and Progress Dots drive the controller, CSS scroll snap delivers free swipe and keyboard scrolling, and a `role="region"` wrapper with Left/Right key handlers makes the whole component operable without a mouse.

Treat accessibility and autoplay as interaction rules, not visual extras. Meaningful `alt` text, labeled buttons, current-slide semantics, and 44×44px hit targets make each control usable, while autoplay stops for focus, pointer interaction, manual navigation, or reduced-motion preferences and restarts only when requested.

Tier image loading by user intent. The first slide ships eagerly, later slides can use native lazy loading, a small adjacent preload window advances as the user navigates, and `srcset` plus `sizes` right-size each request.

Start with the external API and the controller that funnels every input source through one action set; the rest of the design builds on that decision.

Controlled APIs, custom renderers, failed-media recovery, seamless loop clones, virtualization, exact observers and preload scheduling, detailed RTL/caption policy, and a full assistive-technology test matrix are potential follow-ups rather than requirements for the core interview answer.

References

The references below group image carousel sources by theme, so it is easier to map each source back to the part of the article it informed.

Accessibility and UX patterns

This subsection covers the accessibility and user experience foundations for designing a usable carousel.

- [Creating an Accessible Image Carousel](#) for a hands-on tutorial that ties markup, keyboard support, and ARIA together.
- [A Content Slider](#) for an inclusive design walkthrough of slider semantics and interaction design.
- [Designing A Perfect Carousel UX](#) for UX research and patterns that actually keep carousels useful.
- [Carousel pattern | W3C ARIA APG](#) for the canonical ARIA authoring practices governing carousel semantics.

Production case studies

These case studies show how product teams ship carousels and image-heavy surfaces at scale.

- [Building a Faster Web Experience with the postTask Scheduler | Airbnb](#) for visibility-delayed, cancellable, and staggered image preloading.

Performance and lazy loading

This subsection collects the browser APIs and techniques that keep carousels cheap to render and navigate.

- [Intersection Observer API | MDN](#) for detecting when slides enter or leave the viewport to drive prefetching and autoplay pausing.
- [Lazy loading images and iframe elements | web.dev](#) for native `loading="lazy"` guidance that avoids downloading off-screen slides.

Related system design articles

The articles below overlap with the image carousel on media-heavy product pages and media galleries.

- [Photo sharing \(Instagram\)](#) for how a feed of media items uses carousels within a larger image-centric product.
- [E-commerce website \(Amazon\)](#) for how product page image galleries and carousels integrate into a commerce surface.

Exercises

Exercise 1 of 16 Beta

Which requirements define the scoped image carousel before its architecture is chosen? Select all that apply.

- 1

Support desktop, tablet, and mobile interaction.

☐
- 2

Cycle from the final image to the first and from the first image to the final one.

☐
- 3

Require consumers to own the active index and replace every slide and control renderer.

☐
- 4

Receive the complete image list as component configuration before initialization.

☐
- 5

Animate image changes with horizontal movement.

☐



Reveal answer

Check answer ←

Discussions

Normal T

B

I

U

<>

Aa

</>

Type comment here...

0/1000

Post

 Report an issue

  Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

- Get started
- JavaScript functions
- User interface coding
- System design
- Quiz

Guides

Front End Interview Playbook

Study Plans

- 1 Week Plan
- 1 Month Plan
- 3 Months Plan

Company

- Pricing
- Promotions
- Roadmap

- Front End System Design Playbook
- React Interview Playbook
- Behavioral Interview Playbook

- About
- Team
- Contact us
- Become an affiliate
- Careers
- Blog
- Medium
- DEV Community



English (US) ▾